Summary

Object - Oriented Modeling and UML

1.OS 01

Conceptual Modeling

Capturing ideas and concepts in formal that allows other activities reasoning, analysis, prioritization, and design. To better understand a real world system, visualize it as is or as we want it to become, discover and document key decisions specify solutions.

Maximizing accidental complexity by designing software components against abstract service contract and interface rather than concrete, platform-dependent implementations.

Use identifiers that improve Readability and Maintability

Classes: Noun or noun-phrase identifiers formulated in Pascal Casing representing prototypical set instances

Variables: intent-revealing identifiers formulated in lower Camel casing reflecting designated.

Methods: Verb a verb-phrase identifiers denoting operational execution

Validating discrete code units via automated, executable test sites to identify design, fault ensure repeatable functional correctness, document component interfaces and support regression testing frameworks

1.OS 02

UML

Is an object-oriented modeling language (or) more precisely, a collection of modeling languages that is

Modeling elements and Building Blocks:

Structural Elements: Objects, classes, relationships, interfaces, components an execution nodes

Behavioral Elements: Messages, states, transition, and system event

VERZATIL

Escaneado con CamScanner

Organizational and Annotational elements, Packages, structural views explanatory notes and formal specification

Focuses on modeling real-world problem domain entities and concepts independently of software constraints

1.03, 08

Basic Use Cases)

External entities (humans, users, hardware peripherals, or external software systems) that interact directly with the target system to achieve specific operational goals, conventionally denoted by stick figure iconography

Behavioral specification defining required system functionality from an end-users prespective, remaining entirely independent of internal technical implementation details

1.05 09

Class Diagrams)

- Objects
- Classes

Class representation Tri-compartmental rectangular structure demarcating Class Name, Attributes and Operations

Attribute

[visibility] name
[multiplicity)[: type]
[= initial-value] [{property-string)

Operation

[visibility] name [(parameter-vis:)]][: return-type]
[{property-string}]

Types

- Generalization
- Association
- Aggregation
- Composition
- Dependency

Escaneado con CamScanner

1.03.10

Denotes the only point initiating and operational work flow

Represents an atomic step tus, or computational execution within, and activity

Directed edges establishing sequential execution precedence or data exchange dependencies

Control Nodes Structural primitives including Decisions nodes forks, and Joins managing conditional branching an concurrent execution threads

Communication Diagrams
topological graphs illustrating structural collaboration and message passing chains among interacting object instances

Sequence Diagrams
Chronological representation emphasizing the temporal ordering of messages exchanged across object lifelines and execution occurrences

VERZATIL

Escaneado con CamScanner